

Exclusão mútua com semáforos

Um experimento controlado sobre o problema
produtor–consumidor com buffer limitado

*Demonstração empírica de que a exclusão mútua não é garantida sem sincronização explícita,
e passa a ser garantida com semáforos POSIX — com medição do custo em tempo de execução.*

AUTOR

Pietro Prauchner

Linguagem	C11 • POSIX pthreads • semáforos sem_t
Ambiente	WSL2 Ubuntu • gcc 15.2.0 • 6 núcleos
Código-fonte	PPrauchner/SO-Trabalho-1-Semaforos
Data	Setembro de 2026

Resumo

Este trabalho implementa o problema **produtor–consumidor com buffer limitado** em C11, com pthreads e semáforos POSIX, para responder ao que o enunciado pede: provar que o código, ao executar, **não** garante exclusão mútua, e que com semáforos ela passa a ser garantida.

A evidência é numérica. Cada execução compara a soma dos itens produzidos com a soma dos itens consumidos; sob sincronização correta as duas somas são necessariamente iguais, e qualquer divergência é corrupção do recurso compartilhado. O experimento compara **três condições**, executadas 20 vezes cada, sob os mesmos parâmetros e a mesma flag de compilação.

Resultado. Com exclusão mútua (FULL), 0 divergências em 20 execuções. Sem ela, mas com a capacidade do buffer ainda respeitada (NO_MUTEX), 20 divergências em 20 execuções. O custo da correção foi medido: FULL levou cerca de **1,65×** o tempo de NO_MUTEX.

O relatório está organizado nas três partes que o enunciado pede, nesta ordem: a descrição da aplicação (§2), o teste implementado (§3) e a execução do experimento (§4).

Conteúdo

Resumo	1
1 Introdução	3
2 A aplicação	3
2.1 O problema	3
2.2 Estrutura do programa	3
2.3 Parâmetros do experimento	4
2.4 As três condições experimentais	4
2.4.1 Por que três condições, e não duas	5
2.4.2 Por que semáforo binário, e não um mutex	5
3 O teste implementado	5
3.1 Seam 1 — a API do buffer, determinística e sem threads	5
3.2 Seam 2 — o binário, com veredito assimétrico	6
3.3 O que não é medido, e por quê	6
4 A execução do experimento	6
4.1 Ambiente	6
4.2 Resultados	7
4.3 Análise: a exclusão mútua	8
4.4 Análise: o tempo	9
4.5 Achado à parte: a rodada -02	9
5 Conclusão	10
Apêndice A — Como reproduzir	11
Apêndice B — Por que C, e não Java	11

1 Introdução

O enunciado parte de uma observação sobre ambientes multiprocessados: o compartilhamento de memória entre threads, somado à preempção, pode produzir resultados numéricos inconsistentes. Pede então um programa que use threads e semáforos, e que **prove** os dois lados dessa afirmação — a inconsistência sem sincronização, e a consistência com ela.

Provar isso exige mais do que exibir um programa que funciona. Exige um experimento em que a única coisa que muda entre uma execução e outra seja exatamente a presença ou ausência da exclusão mútua, e em que a falha, quando ocorre, não tenha explicação alternativa. É esse cuidado que organiza todo o trabalho:

- **Um recurso compartilhado único e identificável** — o buffer circular — de modo que a corrupção medida tenha origem conhecida (§2.1).
- **Três condições experimentais**, não duas, para que a exclusão mútua possa ser isolada como variável única (§2.4).
- **Parâmetros fixos no código**, para que execuções comparadas lado a lado difiram em uma coisa só (§2.3).
- **Vinte execuções por condição**, porque uma corrida de dados não se testa com uma execução (§3.2).

2 A aplicação

2.1 O problema

Produtor–consumidor com **buffer limitado**. Quatro threads produtoras depositam itens num buffer circular de capacidade fixa; quatro threads consumidoras retiram esses itens e os contabilizam. O buffer é o recurso compartilhado cuja corrupção o experimento mede.

Cada produtor gera itens a partir do próprio índice e de um contador local, de modo que nenhum item depende de estado compartilhado para ser *produzido* — o compartilhamento está só no buffer. Ao final da execução, somam-se os itens que cada produtor gerou (`produced_checksum`) e os que cada consumidor retirou (`consumed_checksum`). Sob sincronização correta, as duas somas são necessariamente iguais.

O acumulador de cada thread vive numa struct própria (`ThreadState`), lida somente depois do `pthread_join`. Isso é deliberado: se os checksums fossem acumulados numa variável global, uma divergência poderia ser atribuída à corrupção do *acumulador* em vez da corrupção do *buffer*. Isolando o acumulador, sobra uma explicação só.

2.2 Estrutura do programa

Tabela 1: Os dois módulos do programa e suas responsabilidades.

Módulo	Responsabilidade
src/buffer.c, src/buffer.h	O buffer circular, os índices de leitura e escrita e os três semáforos POSIX.
src/main.c	Criação e junção das threads, cronometragem da fase concorrente, agregação dos checksums e impressão da linha de resultado.

A separação não é decoração arquitetural: ela existe para que este relatório possa apontar **o arquivo e a função exatos onde a exclusão mútua é decidida**. Toda a decisão mora em `buffer_put` e `buffer_take`, nas linhas que fazem `sem_wait` e `sem_post` sobre o semáforo `mutex`. Fora dali, nenhum código do programa sabe que existe sincronização.

Onde a sincronização é omitida de propósito, o comentário no código diz que a omissão é intencional — sem isso, um leitor razoável leria como bug o que é a variável do experimento.

A escolha de **C em vez de Java**, que o enunciado sugere, está registrada e justificada no Apêndice B.

2.3 Parâmetros do experimento

```
#define N_PRODUCERS 4
#define N_CONSUMERS 4
#define N_ITEMS      100000
#define BUFFER_SIZE  8
```

São constantes no código, não opções de linha de comando; só o modo de sincronização vem de `argv`. Isso é requisito de **reprodutibilidade**: se os parâmetros fossem configuráveis, duas execuções comparadas lado a lado poderiam diferir em mais de uma variável, e a comparação perderia o sentido.

`BUFFER_SIZE 8` é pequeno de propósito. Um buffer pequeno mantém produtores e consumidores disputando os mesmos poucos slots o tempo todo, o que alarga a janela em que a corrida de dados se manifesta. Um buffer grande a esconderia.

2.4 As três condições experimentais

O programa recebe o modo de sincronização em `argv` e o mantém fixo pela execução inteira. Cada modo remove **exatamente uma coisa**:

Tabela 2: Os três modos de sincronização. `empty` e `full` são semáforos contadores (controlam capacidade); `mutex` é o semáforo binário (garante exclusão mútua).

sync_mode	empty / full	mutex	O que é removido
FULL	presente	presente	nada — é a referência
NO_MUTEX	presente	ausente	só a exclusão mútua
NONE	ausente	ausente	capacidade e exclusão mútua

2.4.1 Por que três condições, e não duas

A comparação óbvia seria “sem semáforo nenhum” contra “com semáforos”, e ela seria insuficiente. Sob `NONE`, quando o checksum diverge, existem *duas* explicações concorrentes: os índices se corromperam por falta de exclusão mútua, **ou** o buffer simplesmente estourou — produtores sobreescreveram slots ainda não lidos porque nada os fazia esperar por espaço livre. Um crítico razoável diria “sua divergência é só o buffer transbordando, não tem nada a ver com exclusão mútua”, e estaria certo em duvidar.

`NO_MUTEX` existe para fechar essa saída. Nele os semáforos contadores `empty` e `full` continuam ativos, então a **capacidade do buffer é respeitada**: nenhum produtor escreve sem que haja slot livre, nenhum consumidor lê sem que haja item. A única coisa ausente é o semáforo binário que serializa a atualização dos índices.

`NO_MUTEX` é a condição que isola a exclusão mútua como variável única — e é ela que sustenta a afirmação pedida pelo enunciado. `NONE` permanece no experimento como o extremo ilustrativo, mostrando o que acontece quando nem a capacidade é controlada; mas a prova está em `NO_MUTEX`.

2.4.2 Por que semáforo binário, e não um mutex

A exclusão mútua poderia vir de um `pthread_mutex_t`. Ela vem de um `sem_t` inicializado em 1 porque o enunciado pede semáforos e porque, assim, o contraste fica explícito dentro do mesmo tipo de primitiva: o mesmo `sem_wait` de Dijkstra serve para contar recursos (`empty`, `full`) e para garantir exclusão mútua (`mutex`), e a diferença está no valor inicial e na intenção, não na API.

Isso torna visível no código que **semáforo contador não é mutex** — uma confusão que o próprio resultado de `NO_MUTEX` desfaz.

3 O teste implementado

O teste tem **dois seams** — dois pontos por onde o programa é observado —, e eles respondem a perguntas diferentes.

3.1 Seam 1 — a API do buffer, determinística e sem threads

`tests/test_buffer.c` exercita o contrato público do buffer em uma thread só, com sequências de chamadas que mantêm o buffer sempre entre vazio e `BUFFER_SIZE`, de modo que nenhuma chamada possa bloquear. São cinco testes: ordem FIFO, wrap-around do buffer circular, capacidade, e o comportamento FIFO de `NO_MUTEX` e `NONE` quando não há concorrência.

O propósito é **separar defeito de estrutura de dados de defeito de concorrência**. Se esses testes falham, a sincronização não é a suspeita — o buffer circular está errado como estrutura, e qualquer conclusão sobre semáforos tirada dali seria inválida. Que `NO_MUTEX` e `NONE` também sejam FIFO com uma thread só é o que garante que a divergência observada no seam 2 vem da concorrência, e não de os modos serem estruturalmente diferentes.

3.2 Seam 2 — o binário, com veredito assimétrico

tests/battery.sh executa o binário **20 vezes por modo de sincronização**, lê a linha de resultado de cada execução e conta quantas divergiram. O veredito é **assimétrico**, e é essa assimetria que faz do script um teste de verdade:

Tabela 3: Polaridade do veredito por modo. A assimetria é o que transforma a bateria em teste.

Modo	Polaridade	Reprova quando
FULL	invariante	qualquer execução diverge
NO_MUTEX	corrida de dados	nenhuma execução diverge
NONE	corrida de dados	nenhuma execução diverge

Sob FULL, a afirmação é uma invariante: a exclusão mútua torna a divergência *impossível*, então uma única divergência em vinte execuções a refuta. Sob NO_MUTEX e NONE, a afirmação é o oposto: a corrida deve ser observável, e vinte execuções sem nenhuma divergência significam que a janela é estreita demais nesta plataforma — o que é um achado a relatar, não um bug do programa.

Repetir vinte vezes existe porque **uma corrida de dados não se testa com uma execução**. Uma execução limpa sob NO_MUTEX não provaria ausência de corrida; provaria apenas que naquele escalonamento específico ela não se manifestou.

3.3 O que não é medido, e por quê

A linha de resultado traz produced_checksum, consumed_checksum, difference e time_ms — não traz contagem de *itens perdidos*. A ausência é deliberada: cada consumidor executa exatamente $N_ITEMS / N_CONSUMERS$ chamadas a buffer_take, por quota fixa, em todos os modos. O número de itens retirados é portanto constante por construção, e uma coluna “itens perdidos” seria sempre zero — mediria o laço for, não o buffer.

A perda de item existe, mas se manifesta de outra forma: um item sobrescrito antes de ser lido, ou lido duas vezes, aparece como **divergência de checksum**. O sinal de difference distingue os dois casos: positivo quando itens foram perdidos, negativo quando foram contabilizados em duplicidade.

4 A execução do experimento

4.1 Ambiente

Tabela 4: Ambiente de execução e parâmetros da medição oficial.

Item	Valor
Sistema	WSL2 Ubuntu, kernel 6.18.33.1-microsoft-standard-WSL2
Núcleos disponíveis	6
Compilador	gcc 15.2.0 (Ubuntu 15.2.0-16ubuntu1)

Item	Valor
Flags da medição oficial	-std=c11 -Wall -Wextra -O0 -pthread
Execuções por modo	20

A **flag da medição oficial** é **-O0**, e ela é a mesma para as três condições. Isso é essencial: tempos só se comparam entre binários compilados com a mesma flag. Comparar o -O0 de uma condição com o -O2 de outra não mediria sincronização, mediria o compilador.

Não se usou `volatile` para forçar a corrida a aparecer. `volatile` não dá atomicidade — apenas inibe otimização —, e recorrer a ele faria o experimento medir o compilador em vez do sistema operacional.

4.2 Resultados

Tabela 5: Bateria oficial (-O0), 20 execuções por condição. O checksum produzido é sempre 5.000.050.000.

Condição	Execuções	Divergentes	Mín. (ms)	Média (ms)	Máx. (ms)
FULL	20	0	142,2	159,1	196,1
NO_MUTEX	20	20	79,9	96,4	117,3
NONE	20	20	3,8	4,8	5,9

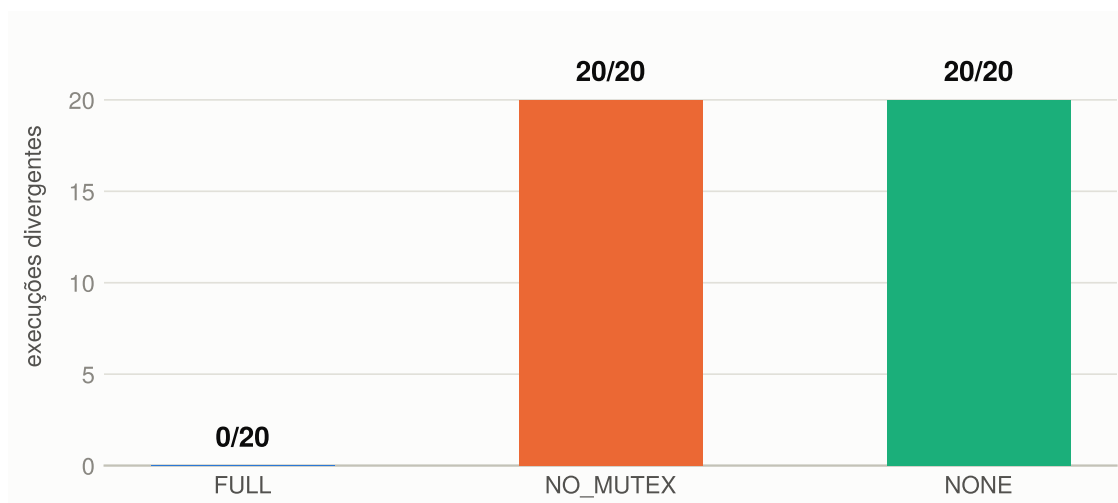


Figura 1: Execuções divergentes em 20 execuções por condição, sob -O0. A exclusão mútua torna a divergência impossível; sem ela, a divergência é a regra, não a exceção.

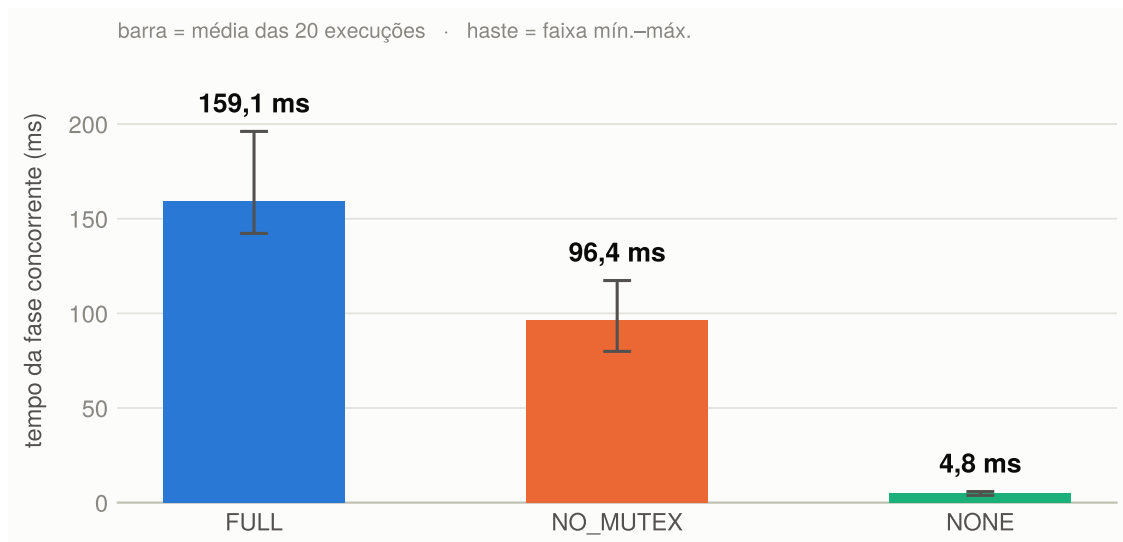


Figura 2: Tempo da fase concorrente por condição, sob -00. A correção tem preço: FULL custa cerca de 1,65× o tempo de NO_MUTEX. NONE é rápido porque não faz o trabalho.

O checksum produzido é sempre 5.000.050.000 — a soma de 1 a 100.000, o mesmo valor em todas as execuções e em todos os modos, porque a geração dos itens não depende de estado compartilhado. É contra esse valor fixo que o consumo é comparado.

Tabela 6: Amostra de execuções individuais, sob -00. As vinte execuções de FULL produziram exatamente a linha mostrada, variando só no tempo. Nas três linhas de NO_MUTEX aparecem os dois sinais de difference — item perdido e item contabilizado em duplicidade.

Condição	consumed_checksum	difference	time_ms
FULL	5.000.050.000	0	150,750
NO_MUTEX	4.995.086.681	+4.963.319	88,870
NO_MUTEX	4.993.756.226	+6.293.774	91,111
NO_MUTEX	5.003.844.293	−3.794.293	91,323
NONE	5.861.826.739	−861.776.739	4,832
NONE	7.418.347.247	−2.418.297.247	3,384

4.3 Análise: a exclusão mútua

Sob FULL, a exclusão mútua é garantida. Vinte execuções, zero divergências, checksum idêntico ao produzido em todas elas. É a metade da afirmação do enunciado que trata do “depois”: com semáforos, a exclusão mútua passa a ser garantida.

Sem ela, não é garantida — e a falha não é rara. Sob NO_MUTEX, as vinte execuções divergiram. Como os semáforos contadores continuavam ativos, o buffer nunca estourou: a capacidade foi respeitada o tempo todo. A única diferença em relação a FULL é a ausência do semáforo binário sobre os índices, e ela basta para corromper o resultado em toda execução.

O mecanismo é visível na aritmética. A linha

```
buffer->write_index = (buffer->write_index + 1) % BUFFER_SIZE;
```

não é uma operação atômica: é uma leitura, um cálculo e uma escrita. Dois produtores podem ler o mesmo `write_index`, escrever no **mesmo slot** — um sobrescrevendo o item do outro antes que qualquer consumidor o leia — e ambos avançarem o índice. O mesmo vale, do lado do consumo, para `read_index`: dois consumidores podem ler o mesmo slot e contabilizar o mesmo item duas vezes.

Os dois sinais de `difference` sob `NO_MUTEX` são justamente esses dois casos. Divergência positiva significa consumo menor que a produção: item sobrescrito antes de ser lido. Divergência negativa significa consumo maior: item contabilizado em duplicidade. Que os dois sinais apareçam entre três execuções mostra que ambas as corridas ocorrem — nos índices de escrita e nos de leitura.

Sob `NONE`, a divergência é de outra ordem de grandeza — cerca de 10^9 contra os 10^6 de `NO_MUTEX`, três ordens de magnitude acima. Faz sentido: sem os contadores, nada impede um produtor de escrever num slot ainda não lido nem um consumidor de reler um slot já consumido, e os consumidores acabam somando repetidamente os poucos valores que por acaso estiverem nos oito slots. O resultado não é apenas errado — é errado de um jeito que não guarda relação com o que foi produzido. Ilustração útil, mas não é ela que prova o ponto sobre exclusão mútua.

4.4 Análise: o tempo

Com `-O0` o compilador não otimiza nada, então **os tempos absolutos não representam o desempenho de código otimizado**. Não é possível afirmar, a partir desta medição, quanto custa um `sem_wait` em unidades absolutas.

O que se pode afirmar é a **razão entre as condições**, porque as três foram compiladas com a mesma flag, executadas na mesma máquina e sujeitas ao mesmo laço:

- `FULL` levou cerca de **1,65x** o tempo de `NO_MUTEX` (159,1 ms contra 96,4 ms de média). Essa diferença é o preço da exclusão mútua correta: serializar os índices custa um `sem_wait` e um `sem_post` adicionais por operação, e sob contenção isso vira bloqueio real de thread.
- `FULL` levou cerca de **33x** o tempo de `NONE` (159,1 ms contra 4,8 ms), o que é esperado: `NONE` não bloqueia nunca, nem por capacidade nem por exclusão mútua. Não é um resultado que favoreça `NONE` — é o tempo de um programa que produz números errados.

A leitura honesta do experimento é essa: **a correção tem custo mensurável**, e o custo é a razão entre `FULL` e `NO_MUTEX` — a única comparação em que apenas a exclusão mútua muda.

4.5 Achado à parte: a rodada -O2

Esta rodada não é a medição oficial. Ela existe porque o nível de otimização é, ele próprio, uma variável que pode interferir na visibilidade da corrida: com `-O2` o gcc pode manter estado compartilhado em registrador e alterar quando — ou se — a corrida se manifesta. Registrar

isso separadamente evita que um resultado do compilador seja confundido com um resultado do sistema operacional.

Tabela 7: Comparação entre as duas rodadas. As divergências não mudaram; os tempos, quase nada.

Condição	Diverg. -00	Diverg. -02	Média -00 (ms)	Média -02 (ms)
FULL	0	0	159,1	164,5
NO_MUTEX	20	20	96,4	96,2
NONE	20	20	4,8	2,9

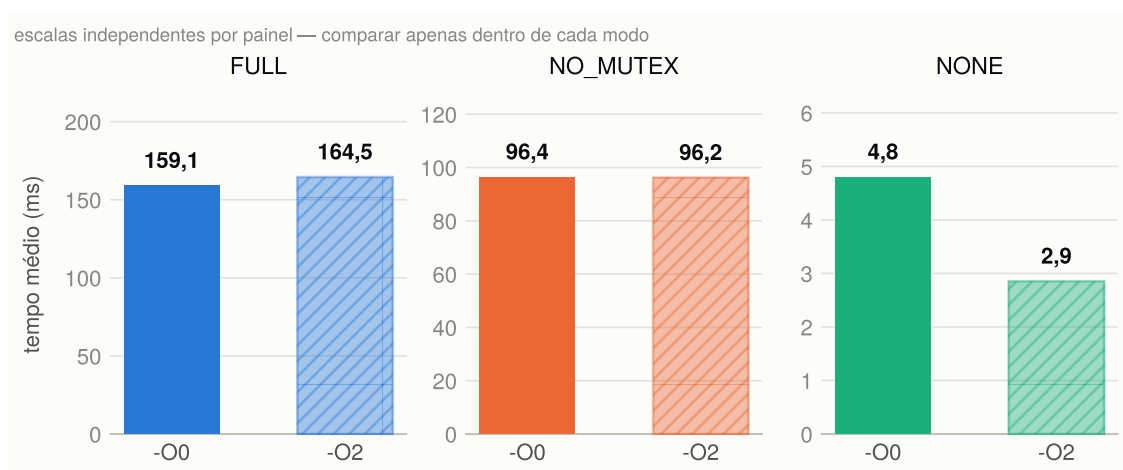


Figura 3: Tempo médio sob -00 e sob -02, por condição. Cada painel tem escala própria: a comparação válida é dentro de cada modo, nunca entre painéis.

Três observações:

1. **A corrida não desapareceu sob -02.** Vinte de vinte execuções divergiram em NO_MUTEX e em NONE, exatamente como sob -00. A conclusão do experimento não depende de o compilador não otimizar — o que dispensa qualquer artifício como `volatile` para mantê-la visível.
2. **FULL e NO_MUTEX praticamente não mudaram de tempo.** Esperado: essas duas condições passam a maior parte do tempo em chamadas de sincronização e em bloqueio de thread, que o compilador não otimiza. O que -02 melhora é código de usuário, e sobra pouco dele aqui.
3. **NONE ficou visivelmente mais rápido** (2,9 ms contra 4,8 ms de média), justamente por ser a única condição dominada por código de usuário simples, sem nenhuma chamada de sincronização a esperar.

Vale repetir o limite: os números desta seção só se comparam entre si. Cruzar o -02 de uma condição com o -00 de outra não diria nada sobre sincronização.

5 Conclusão

O experimento sustenta as duas metades do que o enunciado pede.

Sem exclusão mútua, ela não é garantida. Sob `NO_MUTEX` — com a capacidade do buffer respeitada pelos semáforos contadores e apenas o semáforo binário ausente — as vinte execuções divergiram, e a construção do experimento não deixa explicação alternativa para essa divergência. Não foi preciso forçar nada: nenhum `sleep`, nenhum `volatile`, nenhum ajuste para provocar a corrida.

Com semáforos, ela é garantida. Sob `FULL`, as vinte execuções produziram checksum exato, e a invariante se manteve também na rodada -02.

A correção tem custo, e ele foi medido: cerca de 1,65× o tempo de `NO_MUTEX`, sob -00, comparando binários compilados com a mesma flag. É o preço de um resultado certo em lugar de um resultado rápido e errado.

Apêndice A — Como reproduzir

```
wsl -d Ubuntu make check          # testes da API + bateria -00 (medição oficial)
wsl -d Ubuntu make battery-opt    # rodada -02 (achado à parte)

wsl -d Ubuntu ./bin/prodcons full  # uma execução isolada, por modo
wsl -d Ubuntu ./bin/prodcons no-mutex
wsl -d Ubuntu ./bin/prodcons none
```

Saída da bateria oficial:

```
bateria sobre bin/prodcons – 20 execucoes por modo

mode=full      runs=20  divergent=0  PASS  time_ms min=142.221 mean=159.051 max=196.115
mode=no-mutex  runs=20  divergent=20 PASS  time_ms min=79.909  mean=96.429  max=117.327
mode=none      runs=20  divergent=20 PASS  time_ms min=3.836   mean=4.801   max=5.857

bateria: PASS
```

Saída da rodada -02:

```
bateria sobre bin/prodcons-o2 – 20 execucoes por modo

mode=full      runs=20  divergent=0  PASS  time_ms min=127.942 mean=164.513 max=265.145
mode=no-mutex  runs=20  divergent=20 PASS  time_ms min=82.933  mean=96.193  max=116.359
mode=none      runs=20  divergent=20 PASS  time_ms min=0.705   mean=2.854   max=4.202

bateria: PASS
```

Apêndice B — Por que C, e não Java

O enunciado sugere explicitamente Java, e Java está instalado na máquina — então C é a escolha que precisa de justificativa. Ela está registrada na ADR 0001 (`docs/adr/0001-c-pthreads-em-vez-de-java.md`); em resumo:

- Em C, `sem_wait` e `sem_post` **são** o P e o V que o enunciado atribui a Dijkstra, sem camada intermediária entre o que se escreve e o que se explica aqui. O Semaphore de Java fica sobre o `AbstractQueuedSynchronizer`, uma abstração a mais a justificar.
- O aquecimento do JIT contamina a medição de tempo, que é um dos itens pedidos. Sem rodadas de warmup, mede-se compilação, não sincronização.
- Python foi descartado pelo **GIL**: a corrida é real, mas duas threads nunca executam bytecode simultaneamente — mediríamos contenção de GIL num trabalho cujo enunciado abre falando de ambiente multiprocessado.
- C com a API Win32 foi descartado por acrescentar código específico de plataforma a explicar, sem ganho conceitual sobre POSIX.

O custo aceito é que o programa **não compila no Windows nativo**: edita-se no Windows, compila-se e executa-se no WSL Ubuntu.